ros2 심화 2일차 과제 보고서

21기 예비단원 최동현

1. TF2

-TF란 transform으로 두 좌표계 사이의 관계, 즉 B좌표계가 A좌표계 기준으로 어디에, 어떤 방향으로 있는가 를 나타낸 정보이다. TF2는 이런 관계를 모아서 관리해주는 Ros2 라이브러리이다.

-쿼터니언은 회전을 숫자로 적는 표기 방식이다. 좌표계는 장소이름이다. world, turtle1, base_link처럼 누구 기준인가를 나타낸다. 쿼터니언은 각도를 적는 방식이다. 90도를 1.57rad로 적을 수도 있고, (0,0,0.707, 0.707)로 적을 수도 있는데 그중 하나이다. 도나 라디안처럼 단위/표기법에 가깝다.

-쿼터니언을 사용하는 이유는 roll/pitch/yaw는 적용 순서에 따라 결과가 달라진다. x축으로 돌린 다음 z축으로 돌린 결과와 , z축으로 돌린 다음 x축으로 돌린 결과가 서로 다르다. 결국 짐벌락이 생긴다. 예를 들어 pitch가 90도가 되면 roll과 yaw가 같은 방향의 회전이 돼서 자유도 하나를 잃는다. 계산도 불안정해진다. 근데 쿼터니언에는 이런 문제가 없고,회전 두개를 합치거나 중간 각도를 보간하기도 쉽다. 3D로 도는 로봇에서는 이점이 중요하다.

2. Launch

-launch는 노드 여러 개를 명령 하나로 설정까지 포함해서 한 번에 띄우는 방법이다.

```python
def generate_launch_description():          # ros2 launch가 이 함수를 찾아서 호출
    params = os.path.join(get_package_share_directory('turtle_adv'),
                          'config', 'turtlesim.yaml')
    return LaunchDescription([ Node(...), Node(...), Node(...) ])
```

-generate_launch_description()은 노드를 직접 실행하지 않는다. 무엇을 실행할지를 담은 설명서를 만들어서 돌려줄 뿐, 실제 실행은 ros2 launch의 Launch Service가 한다. 그러나 launch는 시작 한 순서만 보장한다. 준비가 끝났는지는 보장하지 않는다. 노드들은 사실상 동시에 뜨고 준비되는 시점도 제각각이다. 그래서 노드쪽에서 견디게 만들거나(follower의 except TransformException: return, 서비스의 wait_for_service(), 액션의 wait_for_server()) launch쪽에서 순서를 제어하는 방법으로 해결한다. 마지막으로는 lifecycle노드가 unconfigured->inactivate->activate 상태를 거치게해서 외부에서 순서를 정확히 제어하는 방식이다. Nav2가 이 방식으로 제어한다.

-yaml파일은 파라미터를 저장한다. 장점은 코드 수정이나 다시 빌드 없이 값만 바꿀 수 있다는 것이다. launch만 다시 하면 바로 반영된다. 실행 중에는 ros2 param set ~로 바꿀 수 있지만 노드가 껐다 켜면 원본 값으로 돌아간다. 그래서 중간프로젝트를 할때 yaml파일로 파라미터를 저장하고 qt로 그 값들을 즉각적으로 수정해서 반영하기도 하였다.

-그런데 c를 배울때는 yaml파일보다는 json파일을 주로 사용하였는데 왜 yaml파일을 주로 사용하는지 궁금하였다. 일단 ros2 가 파라미터 파일로 yaml만 공식 지원한다. json이나 toml파일을 넘기는 공식 방법은 없다. 그러면 yaml이 파라미터를 저장하는데 사용하는 이유는 먼저 주석을 달 수가 있다. #으로 이 파라미터가 어떤 값을 의미하는지 메모가

가능하다. 또한 사람이 읽고 고치기 쉽다. json파일과 비교하여

```yaml
follower:
  ros__parameters:
    k_linear: 0.5   # 0.8 이상이면 코너에서 튀어나감 (11/4 테스트)
    k_angular: 1.0
```

```json
{"follower": {"ros__parameters": {"k_linear": 0.5, "k_angular": 1.0}}}
```

일단 들여쓰기를 이용하여 파라미터를 구별하여서 각 파라미터를 구분하기 쉽다.

3. Action
-action은 오래 걸리는 일을 시킬 때 쓰는 통신 방식이다. 목표를 보내면 진행 상황을 여러번 받고, 끄탄면 결과를 받는다. 중간에 취소도 할 수 있다. 토픽을 라디오, 서비스를 자판기라고 비유하면 액션은 배달주문이다. 주문하면 조리중, 출발, 도착 5분전 알림이 오고, 마지막에 배달완료, 중간에 주문 취소도 가능한 것이 액션과 비슷하다. 서비스로 “목표 지점까지 가”를 시키면 도착할 때까지 아무 소식 없이 기다려야 하고 취소도 못 한다. 그래서 시간이 걸리고, 중간 상황이 궁금하고, 취소도 할수 있는 액션을 특정 상황에 사용한다. Nav2가 대표적이다.

-action은 전부 비동기 콜백이라서, 기다리는 동안에도 노드가 멈추지 않고 다른 일을 할 수 있다는 장점이 있다. 그래서 action은 서비스 3개(목표 보내기, 취소, 결과 받기)와 토픽 2개(피드백, 상태)를 묶어서 만든 것
4. 과제